

3.1 Write a program to construct a Binary Search tree and perform the following operations: Insertion and Deletion of anode

```
class BST {  
    class Node {  
        int data;  
        Node left, right;  
        Node(int data) {  
            this.data = data;  
        }  
    }  
  
    Node root;  
  
    Node insert(Node root, int data) {  
        if (root == null)  
            return new Node(data);  
        if (data < root.data)  
            root.left = insert(root.left, data);  
        else if (data > root.data)  
            root.right = insert(root.right, data);  
        return root;  
    }  
  
    Node delete(Node root, int key) {  
        if (root == null)  
            return null;  
  
        if (key < root.data)
```

```
        root.left = delete(root.left, key);
    else if (key > root.data)
        root.right = delete(root.right, key);
    else {
        if (root.left == null)
            return root.right;
        else if (root.right == null)
            return root.left;

        root.data = minValue(root.right);
        root.right = delete(root.right, root.data);
    }
    return root;
}
```

```
int minValue(Node root) {
    int min = root.data;
    while (root.left != null) {
        min = root.left.data;
        root = root.left;
    }
    return min;
}
```

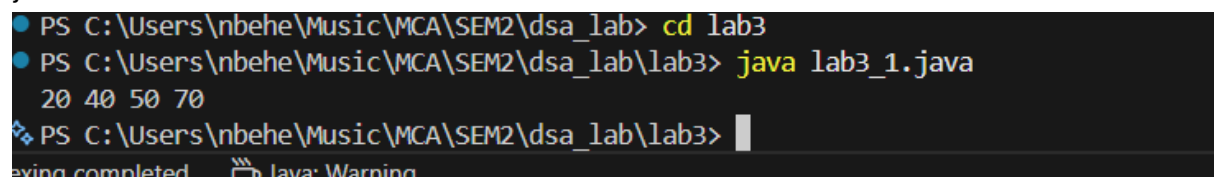
```
void inorder(Node root) {
    if (root != null) {
        inorder(root.left);
        System.out.print(root.data + " ");
    }
}
```

```
        inorder(root.right);
    }
}

public static void main(String[] args) {
    BST tree = new BST();
    tree.root = tree.insert(tree.root, 50);
    tree.insert(tree.root, 30);
    tree.insert(tree.root, 70);
    tree.insert(tree.root, 20);
    tree.insert(tree.root, 40);

    tree.root = tree.delete(tree.root, 30);

    tree.inorder(tree.root);
}
}
```



```
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab> cd lab3
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab3> java lab3_1.java
20 40 50 70
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab3>
```

3.2 Write a program to construct an AVL tree and perform the following operations: insertion, deletion, and searching. class AVLTree {

```
class Node {
    int data, height;
    Node left, right;
    Node(int data) {
        this.data = data;
        height = 1;
    }
}
```

```
}
```

```
Node root;
```

```
int height(Node n) {  
    return n == null ? 0 : n.height;  
}
```

```
int getBalance(Node n) {  
    return n == null ? 0 : height(n.left) - height(n.right);  
}
```

```
Node rightRotate(Node y) {  
    Node x = y.left;  
    Node T2 = x.right;  
  
    x.right = y;  
    y.left = T2;  
  
    y.height = Math.max(height(y.left), height(y.right)) + 1;  
    x.height = Math.max(height(x.left), height(x.right)) + 1;  
  
    return x;  
}
```

```
Node leftRotate(Node x) {  
    Node y = x.right;  
    Node T2 = y.left;
```

```
y.left = x;
x.right = T2;

x.height = Math.max(height(x.left), height(x.right)) + 1;
y.height = Math.max(height(y.left), height(y.right)) + 1;

return y;
}

Node insert(Node node, int key) {
    if (node == null)
        return new Node(key);

    if (key < node.data)
        node.left = insert(node.left, key);
    else if (key > node.data)
        node.right = insert(node.right, key);
    else
        return node;

    node.height = 1 + Math.max(height(node.left), height(node.right));
    int balance = getBalance(node);

    if (balance > 1 && key < node.left.data)
        return rightRotate(node);
    if (balance < -1 && key > node.right.data)
        return leftRotate(node);
```

```
    if (balance > 1 && key > node.left.data) {  
        node.left = leftRotate(node.left);  
        return rightRotate(node);  
    }  
    if (balance < -1 && key < node.right.data) {  
        node.right = rightRotate(node.right);  
        return leftRotate(node);  
    }  
  
    return node;  
}
```

```
Node minValueNode(Node node) {  
    Node current = node;  
    while (current.left != null)  
        current = current.left;  
    return current;  
}
```

```
Node delete(Node root, int key) {  
    if (root == null)  
        return root;  
  
    if (key < root.data)  
        root.left = delete(root.left, key);  
    else if (key > root.data)  
        root.right = delete(root.right, key);  
    else {
```

```
if ((root.left == null) || (root.right == null)) {  
    Node temp = (root.left != null) ? root.left : root.right;  
    if (temp == null)  
        root = null;  
    else  
        root = temp;  
} else {  
    Node temp = minValueNode(root.right);  
    root.data = temp.data;  
    root.right = delete(root.right, temp.data);  
}  
}  
  
if (root == null)  
    return root;  
  
root.height = Math.max(height(root.left), height(root.right)) + 1;  
int balance = getBalance(root);  
  
if (balance > 1 && getBalance(root.left) >= 0)  
    return rightRotate(root);  
if (balance > 1 && getBalance(root.left) < 0) {  
    root.left = leftRotate(root.left);  
    return rightRotate(root);  
}  
if (balance < -1 && getBalance(root.right) <= 0)  
    return leftRotate(root);  
if (balance < -1 && getBalance(root.right) > 0) {
```

```
        root.right = rightRotate(root.right);
        return leftRotate(root);
    }

    return root;
}
```

```
boolean search(Node root, int key) {
    if (root == null)
        return false;
    if (key == root.data)
        return true;
    if (key < root.data)
        return search(root.left, key);
    return search(root.right, key);
}
```

```
void inorder(Node root) {
    if (root != null) {
        inorder(root.left);
        System.out.print(root.data + " ");
        inorder(root.right);
    }
}
```

```
public static void main(String[] args) {
    AVLTree tree = new AVLTree();
    tree.root = tree.insert(tree.root, 10);
}
```



```
tree.root = tree.insert(tree.root, 20);
```

```
tree.root = tree.insert(tree.root, 30);
```

```
tree.root = tree.insert(tree.root, 40);
```

```
tree.root = tree.insert(tree.root, 50);
```

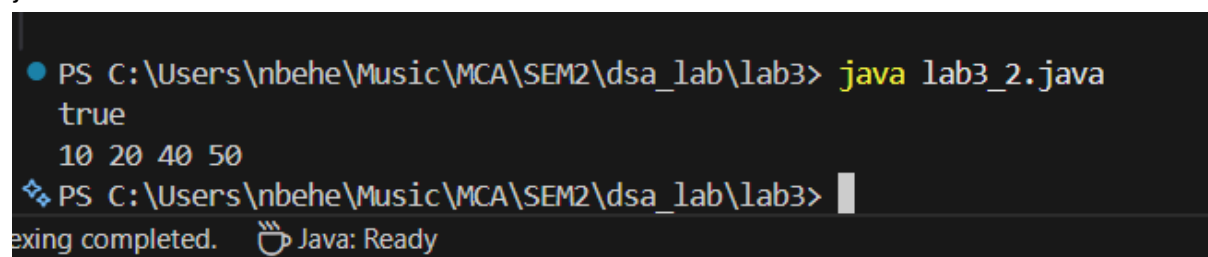
```
tree.root = tree.delete(tree.root, 30);
```

```
System.out.println(tree.search(tree.root, 40));
```

```
tree.inorder(tree.root);
```

```
}
```

```
}
```



```
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab3> java lab3_2.java
true
10 20 40 50
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab3>
```

Execution completed. Java: Ready

3.3 Write a program to construct an AVL tree and perform the following operations: insertion, deletion and searching.

```
class AVLTree {
```

```
    class Node {
```

```
        int key, height;
```

```
        Node left, right;
```

```
        Node(int d) {
```

```
            key = d;
```

```
            height = 1;
```

```
        }
```

```
    }
```

```
Node root;
```

```
int height(Node n) {  
    if (n == null)  
        return 0;  
    return n.height;  
}
```

```
int getBalance(Node n) {  
    if (n == null)  
        return 0;  
    return height(n.left) - height(n.right);  
}
```

```
Node rightRotate(Node y) {  
    Node x = y.left;  
    Node t2 = x.right;  
  
    x.right = y;  
    y.left = t2;  
  
    y.height = Math.max(height(y.left), height(y.right)) + 1;  
    x.height = Math.max(height(x.left), height(x.right)) + 1;  
  
    return x;  
}
```

```
Node leftRotate(Node x) {  
    Node y = x.right;  
    Node t2 = y.left;  
  
    y.left = x;  
    x.right = t2;  
  
    x.height = Math.max(height(x.left), height(x.right)) + 1;  
    y.height = Math.max(height(y.left), height(y.right)) + 1;  
  
    return y;  
}
```

```
Node insert(Node node, int key) {  
    if (node == null)  
        return new Node(key);  
  
    if (key < node.key)  
        node.left = insert(node.left, key);  
    else if (key > node.key)  
        node.right = insert(node.right, key);  
    else  
        return node;  
  
    node.height = 1 + Math.max(height(node.left), height(node.right));  
  
    int balance = getBalance(node);
```

```
if (balance > 1 && key < node.left.key)
    return rightRotate(node);
```

```
if (balance < -1 && key > node.right.key)
    return leftRotate(node);
```

```
if (balance > 1 && key > node.left.key) {
    node.left = leftRotate(node.left);
    return rightRotate(node);
}
```

```
if (balance < -1 && key < node.right.key) {
    node.right = rightRotate(node.right);
    return leftRotate(node);
}
```

```
return node;
}
```

```
Node minValue(Node node) {
    Node current = node;
    while (current.left != null)
        current = current.left;
    return current;
}
```

```
Node delete(Node root, int key) {
```

```
if (root == null)
    return root;

if (key < root.key)
    root.left = delete(root.left, key);

else if (key > root.key)
    root.right = delete(root.right, key);

else {

    if ((root.left == null) || (root.right == null)) {
        Node temp;

        if (root.left != null)
            temp = root.left;
        else
            temp = root.right;

        if (temp == null) {
            root = null;
        } else
            root = temp;
    } else {

        Node temp = minValue(root.right);
        root.key = temp.key;
        root.right = delete(root.right, temp.key);
```

```
    }  
}  
  
if (root == null)  
    return root;  
  
root.height = Math.max(height(root.left), height(root.right)) + 1;  
  
int balance = getBalance(root);  
  
if (balance > 1 && getBalance(root.left) >= 0)  
    return rightRotate(root);  
  
if (balance > 1 && getBalance(root.left) < 0) {  
    root.left = leftRotate(root.left);  
    return rightRotate(root);  
}  
  
if (balance < -1 && getBalance(root.right) <= 0)  
    return leftRotate(root);  
  
if (balance < -1 && getBalance(root.right) > 0) {  
    root.right = rightRotate(root.right);  
    return leftRotate(root);  
}  
  
return root;  
}
```

```
boolean search(Node root, int key) {  
    if (root == null)  
        return false;  
  
    if (root.key == key)  
        return true;  
  
    if (key < root.key)  
        return search(root.left, key);  
    else  
        return search(root.right, key);  
}
```

```
void inorder(Node root) {  
    if (root != null) {  
        inorder(root.left);  
        System.out.print(root.key + " ");  
        inorder(root.right);  
    }  
}
```

```
public static void main(String[] args) {
```

```
    AVLTree tree = new AVLTree();
```

```
    tree.root = tree.insert(tree.root, 30);
```

```
    tree.root = tree.insert(tree.root, 20);
```

```
tree.root = tree.insert(tree.root, 40);
```

```
tree.root = tree.insert(tree.root, 10);
```

```
tree.root = tree.insert(tree.root, 25);
```

```
System.out.print("Inorder: ");
```

```
tree.inorder(tree.root);
```

```
System.out.println("\nSearch 25: " + tree.search(tree.root, 25));
```

```
tree.root = tree.delete(tree.root, 20);
```

```
System.out.print("After Deletion: ");
```

```
tree.inorder(tree.root);
```

```
}
```

```
}
```

```
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab3> java lab3_3.java
Inorder: 10 20 25 30 40
Search 25: true
After Deletion: 10 25 30 40
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab3>
ng completed. Java: Ready
```